Data Structures

Why do we learn data structures?

Hyunjoon Kim, hyunjoonkim@hanyang.ac.kr, Department of Data Science

Software Development

1. Specification or Requirements
2. Design
3. Coding
4. Testing and Debugging
   - Verification: checking correctness

Problem Solving

- Problem solving
  - Algorithms + data structures
  - For efficient algorithm design, design good data structures.

- Algorithm
  - A finite set of instructions that accomplishes a particular task
- Data structure
  - A particular way of storing and organizing data in a computer so that it can be efficiently used

Algorithm: Factorial

- $n! = 1 \cdot 2 \cdot 3 \cdot \cdots \cdot n$

```
int Fact(n){
    int result = 1;
    for(int i = 1; i <= n; ++i){
        result = i * result;
    }
    return result;
}
```

Two Types of Data

- Two types depending on the organization of the elements
  - Linear types: elements are accessed in a sequential order but sequentially storing all elements is not compulsory
    - linked lists, arrays
  - Non-linear types: elements are stored/accessed in a non-linear order.
    - hash tables

Software Development

1. **Specification or Requirements**
2. **Design**
3. **Coding**
4. Testing and Debugging
   - Verification: checking correctness

Abstract Data Type (ADT)

A collection of data together with

a set of operations on the data

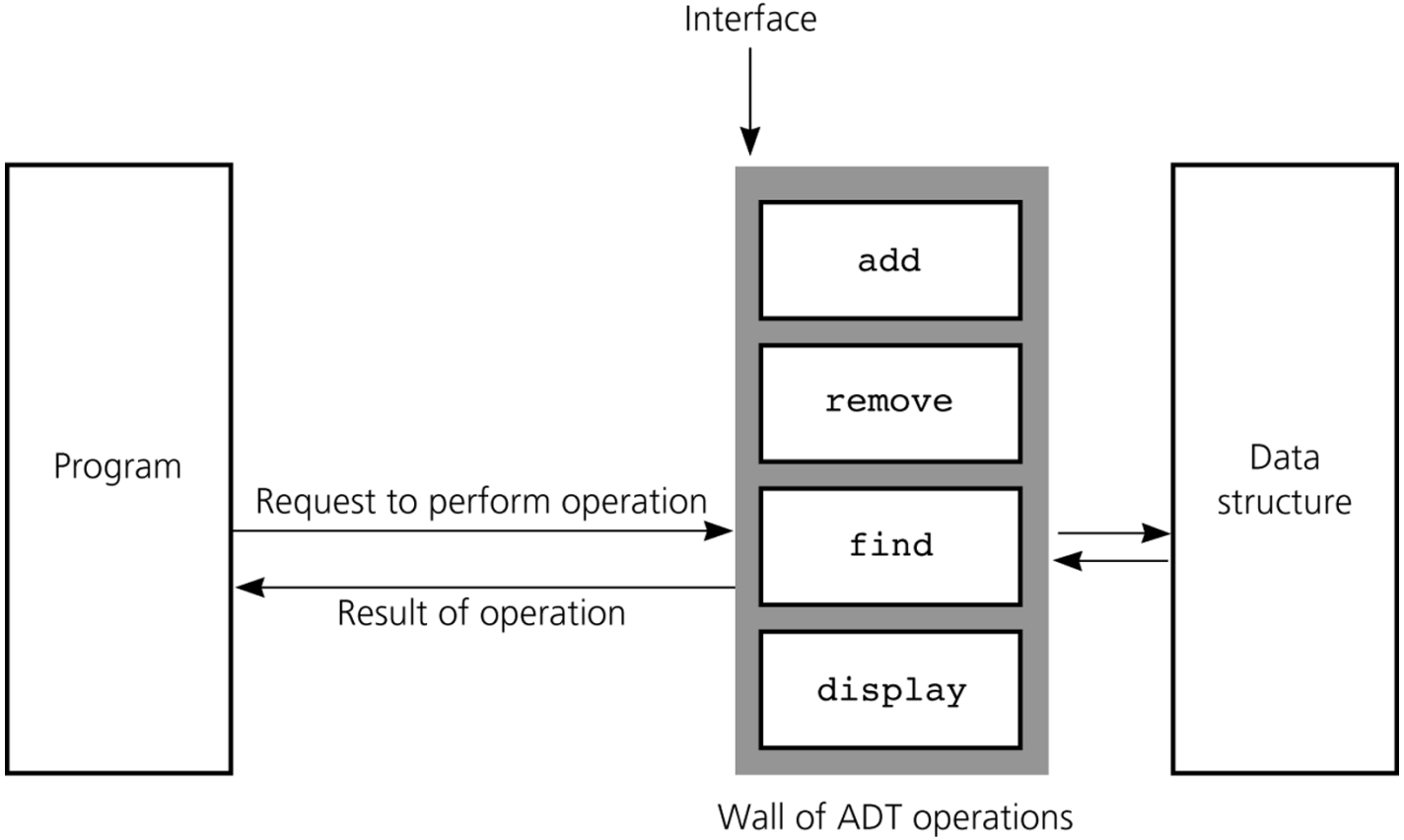
Data Abstraction

- What you can do to a collection of data, not how you do it.
  - The specification of objects (a collection of data) and their operations is separated from the implementation of the objects and operations

- Specification of ADT includes
  - Function name
  - Argument types
  - Result types
  - Description of what a function does

ADT

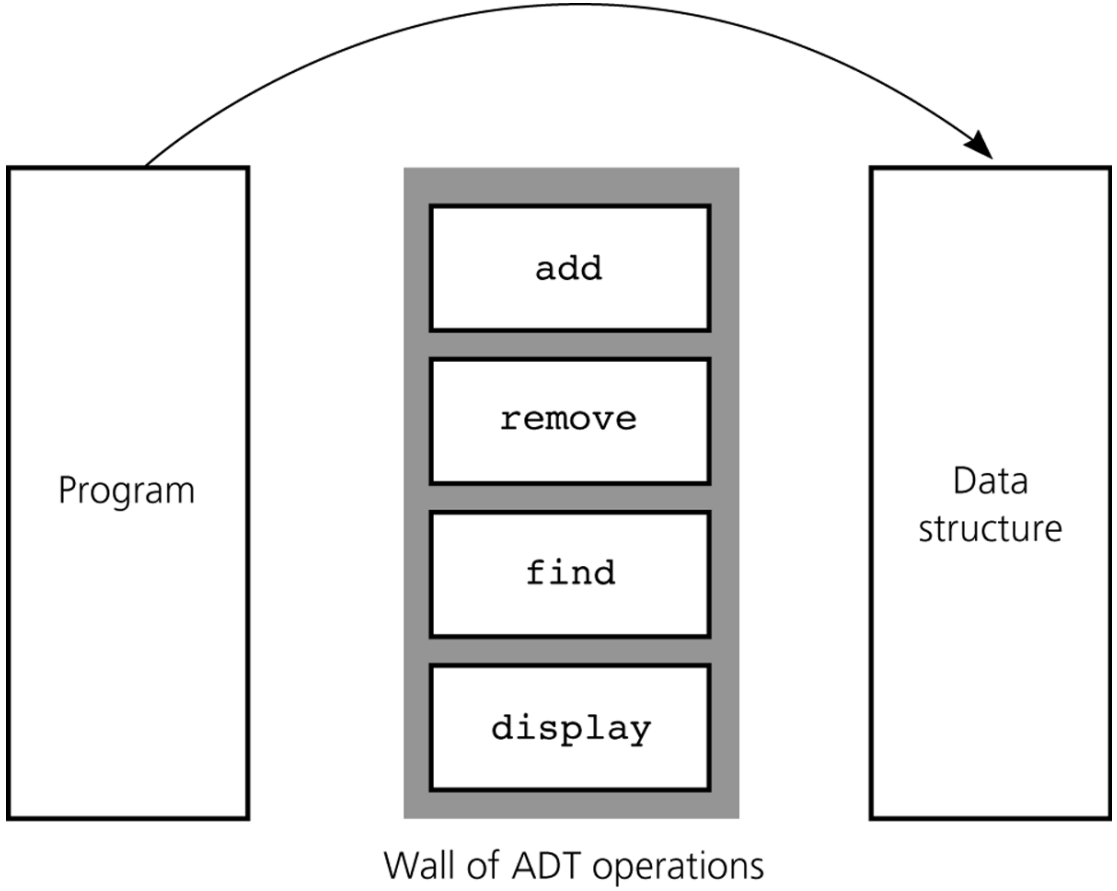
- ADT operations isolate a data structure from the program



Carrano and Prichard, Data Abstraction and Problem Solving with JAVA: Walls and Mirrors

Bad Design

- Direct access to the data structure



Carrano and Prichard, Data Abstraction and Problem Solving with JAVA: Walls and Mirrors

Example: The Stack ADT

```
class Stack:
    def isEmpty(self):
        # Return True if the stack is empty
    def push(self, e):
        # Add element e to the top of the stack
    def top(self):
        # Return (but do not remove)
        # the element at the top of the stack
        # Raise exception if the stack is empty
    def pop(self):
        # Remove and return the element
        # from the top of the stack (i.e., LIFO)
        # Raise exception if the stack is empty
```

Stack instance S supports the methods:

- S.push(e)
- S.pop()
- S.top()
- S.isEmpty()
- len(S)

Example: Credit Card

- UML diagram in object-oriented programming

Class:	CreditCard	
Fields:	_customer _bank _account	_balance _limit
Behaviors:	get_customer( ) get_bank( ) get_account( ) make_payment(amount)	get_balance( ) get_limit( ) charge(price)

Recursive Algorithm

An algorithm that calls itself for subproblems

Recursive Algorithms

- Algorithms that call itself for subproblems
  - The subproblems are of exactly the same type as the original problem (mirror images)
  - Provide a simple look for a complicated problem

Recursive Algorithm: Factorial

- $n! = n \cdot (n-1) \cdot \cdots \cdot 2 \cdot 1$

  $= n \cdot (n-1)!$

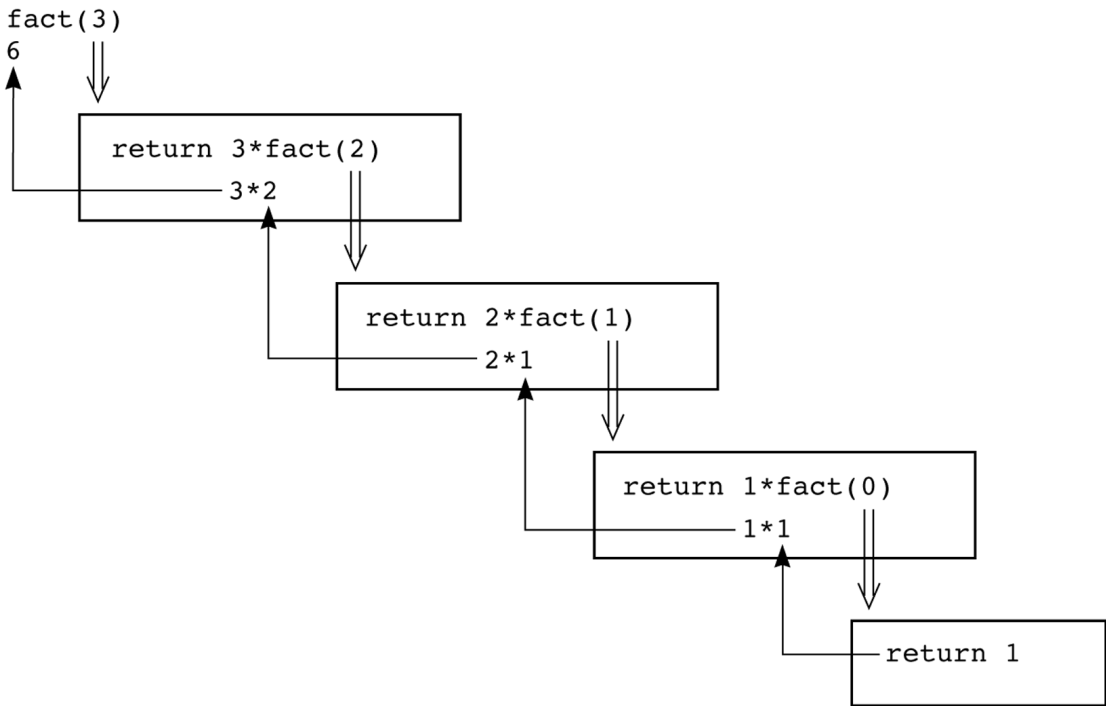
- Recursive structure of factorial

$$Fact(n) = \begin{cases} 1 & if\ n = 0 \\ n \cdot Fact(n-1) & if\ n > 0 \end{cases}$$

```
int Fact(int n){
    if(n == 0) // Base case
        return 1;// Terminate
    else // Recursive case
        return n * Fact(n-1);
}
```

Keys of Recursive Algorithm

- Recursive case
  - Define the problem in terms of smaller problems of the same type
- Base case
- Termination (by base case)



Recursive Algorithm: Fibonacci Sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34,…
- Recurrence of Fibonacci sequence

$$Fib(n) = \begin{cases} 1 & if\ n = 1\ or\ n = 2 \\ Fib(n-1) + Fib(n-2) & if\ \ n > 2 \end{cases}$$

```
int Fib(int n){
    if(n <= 2) // Base case
        return 1;// Terminate
    else // Recursive case
        return Fib(n-1) + Fib(n-2);
}
```